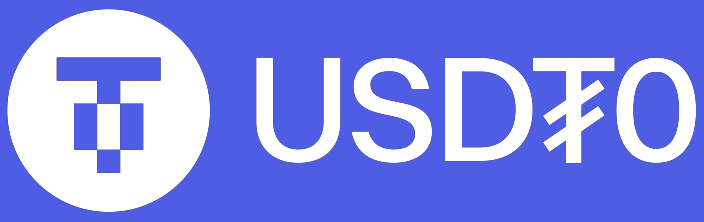
OpenZeppelin | security

Everdawn USDT0 IOTA OFT Adapter Configuration Review



October 9, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 5

Approach 5

Deployment Verification 6

- Core Deployment Identifiers 6
- Bytecode Verification 6
- Deployment Transaction Summary 7
- Ownership and Initialization 9
- Ethereum Pathway Configuration 10
- Isolation From the Other USDT0 Deployments 11
- Message History and Locked USDT 12

Security Model and Trust Assumptions 13

Conclusion 15

Appendix 16

- Issue Classification 16

Summary

Type	Stablecoins
Timeline	From 2026-10-09 To 2026-10-09
Languages	Solidity

Total Issues	0 (0 resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	0 (0 resolved)
Low Severity Issues	0 (0 resolved)
Notes & Additional Information	0 (0 resolved)
Client Reported Issues	0 (0 resolved)

Scope

This report provides a bytecode and configuration review of the USDT0 OFT adapter for IOTA L1 that is deployed on Ethereum mainnet.

The review covered the `OFT-IOTA-production` deployment recorded in `deployments/ethereum-mainnet/OFT-IOTA-production.json` of the Everdawn-Labs/usdt0-oft-contracts repository, together with its implementation, proxy, and ProxyAdmin deployment files. The intended configuration was taken from the `scripts/iota-ethereum-wire.ts` wiring script and the `wiring_batches/iota-1.json` wiring batch of the same repository. The wiring batch was compared with the LayerZero wire dump gist, pinned at revision `a154bb9`.

The contracts and the configuration on IOTA L1 were outside the scope of this review. The previous IOTA deployment review recorded the main USDT0 adapter as the Ethereum peer of the IOTA OFT. Therefore, the change of that peer to this adapter on IOTA L1 was not reviewed. The IOTA peer was only read back to confirm that the pathway is open in both directions.

On-chain state was read on 2026-10-09, at Ethereum block `26154989`. All results describe the state at that time.

System Overview

USDT0 uses the LayerZero Omnichain Fungible Token (OFT) standard to move USDT between chains. On Ethereum, an OFT adapter locks USDT, and OFT contracts on the other chains mint and burn USDT0 against it. The `OFT-IOTA-production` contract is a second, dedicated adapter for the pathway to IOTA L1. The main USDT0 adapter (`0x6C96dE...1dee`) keeps backing the rest of the USDT0 deployments.

OFT Adapter on Ethereum: The `OAdapterUpgradeable` contract extends LayerZero's `OFTAdapterUpgradeable` and runs behind an OpenZeppelin `TransparentUpgradeableProxy` with its own ProxyAdmin. A transfer to IOTA locks USDT in the adapter and sends a LayerZero message to IOTA. A transfer from IOTA releases the corresponding USDT to the recipient. USDT and the shared decimals both use 6 decimals, so amounts are not rounded.

LayerZero Pathway: Messages travel between Ethereum (endpoint ID `30101`) and IOTA L1 (endpoint ID `30423`). On Ethereum, they go through the ULN302 message libraries. The adapter pays three DVNs to verify its outgoing messages and requires the same three DVNs to verify incoming ones: USDT0, LayerZero Labs, and Canary Sponsored. The LayerZero executor then delivers each message on the destination chain.

Approach

The review consisted of the following checks:

1. Bytecode equivalence of the implementation, proxy, and ProxyAdmin with the repository, its compiled artifacts, and the public explorers.
2. Integrity of the deployment transactions and of the initialization.
3. Control of the owner, delegate, and ProxyAdmin roles, and the configuration of the USDT0 Safe.
4. Ethereum-side LayerZero configuration against the wiring script, the wiring batch, the LayerZero wire dump, and LayerZero's published addresses, together with the state of the DVN and executor contracts.

5. Absence of unexpected configuration, based on the complete event history since deployment.
6. Isolation of the adapter from the other USDT0 deployments, the message history, and the USDT that the adapter holds.

Deployment Verification

Core Deployment Identifiers

Role	Address
OFT adapter proxy (`OFT-IOTA-production`)	`0xAEf027...3414`
Implementation (`OAdapterUpgradeable`)	`0x1ab288...1D99`
ProxyAdmin (`DefaultProxyAdmin-IOTA-Adapter`)	`0xa7D54c...Ae52`
USDT0 Safe	`0x4DFF9b...0bf8`
Deployer	`0x1a6362...d154`
Inner token (USDT)	`0xdAC17F...1ec7`
LayerZero EndpointV2	`0x1a4407...728c`
SendUln302	`0xbB2Ea7...dCe1`
ReceiveUln302	`0xc02Ab4...24C2`
Executor	`0x173272...3059`
IOTA OFT package (peer)	`0xe6a11e...5902`

Bytecode Verification

Contract	Reference	Result
Implementation	`OAdapterUpgradeable` compiled with solc `0.8.22`, 200 optimizer runs, and the `shanghai` EVM version. The runtime code is byte-identical, including the metadata hash.	✓ Confirmed

Contract	Reference	Result
Implementation immutables	`endpoint` is EndpointV2, `innerToken` is USDT, and `decimalConversionRate` is `1`	✓ Confirmed
Proxy	hardhat-deploy `TransparentUpgradeableProxy` artifact	✓ Confirmed
ProxyAdmin	hardhat-deploy `ProxyAdmin` artifact	✓ Confirmed
Creation transactions	Creation code followed by the constructor arguments of the deployment files, for all three contracts	✓ Confirmed
Compiler input	The 68 sources of the deployment compiler input are identical to the repository and to the installed packages	✓ Confirmed
Explorer verification	Exact creation and runtime matches on Sourcify and Blockscout for all three contracts	✓ Confirmed

The compiler input uses `@layerzerolabs/oft-evm-upgradeable` 3.0.0, `@layerzerolabs/oapp-evm-upgradeable` 0.1.0, `@layerzerolabs/oft-evm` 3.1.0, and `@layerzerolabs/oapp-evm` 0.3.0. It also uses `@layerzerolabs/lz-evm-protocol-v2` 3.0.160 and OpenZeppelin Contracts and Contracts Upgradeable 5.2.0. These sources were also compared with the published npm packages. The main USDT0, XAUt0, and USAT adapters on Ethereum compile the same `OFTCoreUpgradeable` source. Newer releases of these packages were available when this adapter was deployed, and the LayerZero integration checklist recommends using the latest ones. Their only security-relevant change is an overflow check in `_toSD`: the deployed version silently truncates an amount above `type(uint64).max` in shared decimals. The truncation needs a transfer of about 18.4 trillion USDT, so this adapter cannot reach it. A future deployment for a token with a larger supply could be affected. Future deployments can use the latest LayerZero packages, or document why the versions are pinned, such as to keep the bytecode identical to the other USDT0 adapters. The proxy and ProxyAdmin bytecode is identical to that of the main USDT0 adapter.

Deployment Transaction Summary

The configuration transactions were reconstructed from the events of the adapter, the LayerZero endpoint, both ULN302 libraries, and the ProxyAdmin, from deployment until block `26154989`. The deployer sent all of them, and all of them succeeded.

Block (date)	Transaction	Operation
`25839030` (2026-08-26)	`0x10640f...530a`	Deploy the ProxyAdmin, owned by the USDT0 Safe
`25839031` (2026-08-26)	`0x99660e...22bf`	Deploy the implementation
`25839032` (2026-08-26)	`0xad5e23...9e33`	Deploy the proxy and call `initialize` with the deployer as owner and delegate
`25989129` (2026-09-16)	`0x315163...4fdf`	`setPeer` (wiring batch transaction 1)
`25989130` (2026-09-16)	`0x61c3a3...9e9a`	`setSendLibrary` (wiring batch transaction 2)
`25989131` (2026-09-16)	`0x88b303...e6f0`	`setReceiveLibrary` (wiring batch transaction 3)
`25989132` (2026-09-16)	`0x5276bd...8b18`	`setConfig` on SendUln302 (wiring batch transaction 4)
`25989138` (2026-09-16)	`0xa3e5bd...cb90`	`setConfig` on SendUln302, repeated with identical calldata
`25989160` (2026-09-16)	`0xaaaf46...6069`	`setConfig` on ReceiveUln302 (wiring batch transaction 5)
`25989161` (2026-09-16)	`0x97f0c8...3db3`	`setEnforcedOptions` (wiring batch transaction 6)
`25989162` (2026-09-16)	`0xd5564f...c9da`	`setDelegate` to the USDT0 Safe
`25989163` (2026-09-16)	`0x2da50f...e16c`	`transferOwnership` to the USDT0 Safe

Each wiring transaction is byte-identical to the corresponding transaction of `wiring_batches/iota-1.json`. That batch is in turn identical to the Ethereum transactions of the LayerZero wire dump. The peer was set before the libraries and the DVN configuration, which is the order that LayerZero's wiring tool generates. The LayerZero OFT documentation states that `setPeer` opens the pathway. The send and receive configurations were set in blocks `25989132` and `25989160`. Until then, the open pathway used the

endpoint defaults for IOTA L1: 15 send and 5 receive confirmations, verified by the LayerZero Labs, Horizen, and Nethermind DVNs. These defaults have not changed since the deployment. No message was sent through the adapter before the configuration was complete, so the deployment was not affected. Future deployments can avoid this window by setting the peer in the last wiring transaction, or by executing the whole wiring batch atomically through a single Safe transaction. The repeated `setConfig` transaction writes the same values again and does not change the final state.

The wire dump suggests submitting the Ethereum transactions through the USDT0 Safe. Instead, the deployer executed them and then transferred the owner and delegate roles to the Safe. This leads to the same final state. No other configuration change was found. The adapter was never upgraded, and no other peer, message inspector, or pre-crime contract was ever set. No inbound message was skipped, nilified, or burnt. The USDT0 Safe emitted no events after the deployment.

Ownership and Initialization

Check	Expected	Result
Proxy implementation (EIP-1967 slot)	Implementation (`0x1ab288...1D99`)	✓ Confirmed
Proxy admin (EIP-1967 slot)	ProxyAdmin (`0xa7D54c...Ae52`)	✓ Confirmed
ProxyAdmin `owner`	USDT0 Safe (`0x4DFF9b...0bf8`)	✓ Confirmed
Adapter `owner`	USDT0 Safe (`0x4DFF9b...0bf8`)	✓ Confirmed
Endpoint `delegates` for the adapter	USDT0 Safe (`0x4DFF9b...0bf8`)	✓ Confirmed
Deployer privileges	None left	✓ Confirmed
Implementation initializers	Disabled, `initialize` reverts with `InvalidInitialization`	✓ Confirmed

Check	Expected	Result
Proxy initialization	Version `1`, `initialize` reverts with `InvalidInitialization`	✓ Confirmed
`token` and `endpoint`	USDT and EndpointV2	✓ Confirmed
`approvalRequired`, `sharedDecimals`, and `decimalConversionRate`	`true`, `6`, and `1`	✓ Confirmed
`msgInspector` and `preCrime`	Unset	✓ Confirmed
USDT0 Safe threshold and owners	3-of-5, with the owners listed in the wiring script	✓ Confirmed
USDT0 Safe modules and guard	None	✓ Confirmed

Ethereum Pathway Configuration

Check	Expected	Result
Peer for endpoint ID `30423`	IOTA OFT package (`0xe6a11e...5902`)	✓ Confirmed
Peers for other endpoint IDs	Unset for all other LayerZero V2 endpoint IDs in the mainnet range of LayerZero's metadata (186 IDs), including IOTA EVM (`30284`)	✓ Confirmed
Send library	SendUln302, set explicitly	✓ Confirmed
Receive library	ReceiveUln302, set explicitly, with no grace period for a previous library	✓ Confirmed
Send ULN configuration	`65` confirmations, three required DVNs, no optional DVNs	✓ Confirmed
Receive ULN configuration	`1500` confirmations, three required DVNs, no optional DVNs	✓ Confirmed

Check	Expected	Result
Executor configuration	Executor (`0x173272...3059`), maximum message size `10000`	✓ Confirmed
Enforced options for message types 1 and 2	`lzReceive` gas `5000`, no value, no `lzCompose` option	✓ Confirmed
Infrastructure addresses	Match LayerZero's metadata and `@layerzerolabs/lz-evm-sdk-v2`	✓ Confirmed
DVNs and executor	Not paused, the access list allows the adapter, and the IOTA destination is configured	✓ Confirmed
`quoteSend` to IOTA	Succeeds with a non-zero fee	✓ Confirmed

For reference, the peer table of the IOTA OFT maps endpoint ID `30101` to this adapter, so the pathway is open in both directions.

The required DVNs on Ethereum are the following:

DVN	Address	Signer quorum
USDT0	`0x3b0531...0dD2`	1 of 2
LayerZero Labs	`0x589dED...236b`	2 of 3
Canary Sponsored	`0xF9D2C0...E12C`	1 of 1

LayerZero's metadata lists the Canary Sponsored DVN. On Ethereum, it has the same bytecode, the same signer, and the same quorum as the Canary DVN.

Isolation From the Other USDT0 Deployments

Check	Expected	Result
Main USDT0 adapter peer for endpoint ID `30423`	Unset	✓ Confirmed

Check	Expected	Result
USDT0 OFT peers for endpoint ID `30423` on 20 other chains	Unset	✓ Confirmed
Paths that the adapter accepts (`allowInitializePath`)	Only endpoint ID `30423` from the IOTA OFT package	✓ Confirmed

None of the queried USDT0 contracts has an IOTA peer, so a transfer to IOTA through any of them reverts on the source chain. The USDT0 OFT on MP1 could not be queried, because no public RPC endpoint for MP1 was available.

Message History and Locked USDT

Check	Expected	Result
Messages through the adapter (LayerZero Scan)	321 from Ethereum to IOTA and 156 from IOTA to Ethereum, all delivered	✓ Confirmed
`OFTSent` and `OFTReceived` events	321 and 156, matching LayerZero Scan	✓ Confirmed
USDT held by the adapter	USDT locked by `OFTSent` minus USDT released by `OFTReceived` (241,796.179494 USDT)	✓ Confirmed
USDT allowances granted by the adapter	None (no `Approval` event with the adapter as owner)	✓ Confirmed

One message from IOTA, received on 2026-09-18, carried a compose call addressed to an externally owned account. The adapter released the 0.000001 USDT of that message to the account, and the two `lzCompose` attempts of the executor failed because the account has no code. This does not affect the adapter or the other messages. LayerZero Scan also lists one zero-amount message from the IOTA OFT to the main USDT0 adapter, sent on 2026-04-07, before this adapter was wired. It remains blocked because the main USDT0 adapter has no IOTA peer.

Security Model and Trust Assumptions

USDT0 Safe: The USDT0 Safe (`0x4DFF9b...0bf8`) is the owner of the adapter, its endpoint delegate, and the owner of its ProxyAdmin. As the owner, it can set peers, enforced options, the message inspector, and the pre-crime contract. As the delegate, it can change the libraries and the DVN and executor configuration. It can also skip, clear, nilify, or burn inbound messages. As the ProxyAdmin owner, it can upgrade the adapter to arbitrary code. This gives it control over all locked USDT. Upgrades take effect immediately, because the ProxyAdmin has no timelock. The Safe requires 3 of 5 signatures and has no modules or guard. If the DVNs verify a forged message, only the Safe can stop it, by skipping or nilifying the message before the executor delivers it. The Safe is trusted to keep the Ethereum configuration consistent with the IOTA side and not to misuse these powers.

DVN Operators: The DVN operators are not on-chain roles of the adapter. All three required DVNs must verify each message, and no optional DVNs are configured. A forged message would therefore need the verification of all three DVNs. If one DVN stops verifying, the pathway halts until its configuration changes. On Ethereum, the USDT0 DVN accepts 1 of its 2 signers, and the Canary Sponsored DVN has a single signer. A change of signers or quorum on a DVN needs the signatures of its current quorum, submitted by an admin of the DVN contract. Every admin of the USDT0 and Canary Sponsored DVN contracts is also an admin of the LayerZero Labs DVN contract. The admins set the destination gas and fee parameters, but they cannot verify messages or replace signers on their own. No address holds the default admin role of the DVN contracts, so the DVNs cannot be paused. A DVN can also be misled through its RPC infrastructure, as in the KelpDAO incident. The DVN operators are trusted to verify only messages that were sent on the source chain, to use independent infrastructure, and to keep verifying.

Tether: Tether is the owner of USDT (`0xC6CDE7...a828`) and is not a role of the adapter. It can enable a transfer fee of less than 0.2%, capped below 50 USDT per transfer. If a fee is enabled, the adapter receives less USDT than the amount that it reports to IOTA. The USDT0 supply on IOTA would then exceed its backing. Tether can also blacklist the adapter and destroy its USDT balance. This would leave the USDT0 on IOTA unbacked. Tether is trusted not to use these controls against the adapter.

Executor: The LayerZero executor is not an on-chain role of the adapter. It is trusted to deliver verified messages on the destination chain with the gas set by the enforced options. If the executor does not deliver a message, the transfer is delayed.

LayerZero: The LayerZero endpoints and ULN302 libraries on both chains are trusted to operate as specified. Their addresses match LayerZero's published deployments.

Lockbox Separation: The previous IOTA package audit assumed that USDT0 has a single OFT adapter. This deployment adds a second USDT lockbox on Ethereum, which serves only the pathway to IOTA. The two lockboxes serve disjoint pathways. On Ethereum, this adapter only accepts and sends messages for IOTA, and the main USDT0 adapter has no IOTA peer. A send to the wrong pathway reverts at the source with `NoPeer`, so neither lockbox can release USDT that the other one locked. If a pathway between IOTA and another USDT0 deployment is opened, USDT0 backed by one lockbox could be redeemed from the other lockbox. If the main USDT0 adapter ever set the IOTA OFT as its peer, the blocked zero-amount message from the IOTA OFT to that adapter would also be delivered. The USDT0 Safe and the IOTA multisig (`0xfb11bd...9174`) are trusted to keep the pathways disjoint, or to move the corresponding backing before any such change.

Conclusion

The `OFT-IOTA-production` adapter is a dedicated USDT lockbox on Ethereum for the pathway to IOTA L1, separate from the main USDT0 adapter. OpenZeppelin reviewed its bytecode, deployment transactions, privileged roles, and LayerZero configuration on Ethereum mainnet.

The deployed bytecode matches the repository and the public explorers, and the USDT0 Safe holds the owner, delegate, and ProxyAdmin roles. On Ethereum, the configuration matches the wiring script, the wiring batch, and the LayerZero wire dump, and the event history shows no other configuration change. The adapter only accepts and sends messages for IOTA, every message through it was delivered, and the main USDT0 adapter has no IOTA peer. No issues were identified. For future deployments, setting the peer after the security configuration and using the latest LayerZero packages would align the wiring and the build with LayerZero's guidance.

The adapter is only as safe as the DVNs that verify its messages. On Ethereum, a forged message needs three DVNs from different providers. However, two of them accept a single signer key, and overlapping sets of addresses administer the contracts of all three. The separation between the two lockboxes also depends on the peers that the USDT0 Safe and the IOTA multisig currently configure. If the connectivity of IOTA changes in the future, the corresponding backing should move at the same time.

OpenZeppelin thanks the Everdawn team for its cooperation throughout the review.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.